

HANDOVER DOCUMENT — FPGA Acoustic Drone Detection Project

Purpose of this file: if you are an AI assistant reading this with no memory of prior conversations, this document gives you everything needed to understand the project's current state and continue it correctly. Read this in full before touching any file. The companion `PROJECT_REPORT.md` has the complete session-by-session history if you need forensic detail on any specific decision; this document is the faster-to-read summary optimized for "get up to speed and continue."

1. What this project is

An FPGA hardware accelerator (target: Terasic DE2-115, Cyclone IV EP4CE115) that classifies short audio clips as **drone** or **ambient** (background noise), based on a neural network trained in Python and converted into synthesizable Verilog RTL.

Important terminology note: this is called an "SNN" project by the original brief, but the *trained model itself* has no spiking neuron model (no LIF, no membrane potential, no threshold/leak/reset) — it's an ordinary sigmoid/softmax network trained with backprop, using a published "ANN-to-SNN weight balancing" technique (Diehl et al. 2015). What IS genuinely spiking: the **input encoding** (real LFSR-driven stochastic rate coding, bit-matched to the Python reference) and the **hardware compute style** (event-driven — `spike_gated_mac.v` skips all compute for non-firing neurons, a real resource/latency saving, not just naming). Accurate description: "rate-coded, spike-driven neuromorphic accelerator." Do not describe the neurons themselves as

LIF — they aren't, and claiming otherwise would be factually wrong about a model whoever continues this project may need to defend under questioning.

2. The complete data/software pipeline (already existed before RTL work began)

1. **Audio:** 16kHz mono WAV, drone or ambient class, from `saraaalemadi/DroneAudioDataset` (real dataset, ~11,700 files, ~1,332 drone / ~10,372 ambient).
2. **STFT feature extraction** (`audio_to_spike.py`): 256-point STFT, 192-sample overlap (64 hop), Hann window, `boundary=None`, `padded=False` (deliberately matches streaming-hardware framing, not scipy's padded default). Magnitude → dB → normalized against a **fixed** reference (-60dB floor, 0dB ceiling — NOT per-file min/max, which was tried and rejected for destroying loudness information). Downsampled via average pooling to **32×20 = 640 values**, each in [0,1].
3. **Deterministic rate encoding**
(`deterministic_rate_encode()` in `audio_to_spike.py`): each of the 640 feature values becomes a firing *probability* (`threshold_int = floor(clip(feature*0.9, 0, 1) * 65535)`), **note: floor/truncating cast, not round** — this exact detail was load-bearing for RTL bit-exactness). A single shared 16-bit Fibonacci LFSR (seed `0xACE1`, taps 15/13/12/10, one advance per neuron per timestep, row-major iteration order) is compared against each neuron's threshold, once per timestep, for 50 timesteps → a (50, 32, 20) binary spike tensor. **This LFSR is the actual freeze contract between Python and RTL** — same algorithm, same seed, same iteration order, verified bit-exact.
4. **Golden dataset generation** (`validate_encoding.py`): runs the above on real files, saves `<name>_feature.npy` (32×20 float) and `<name>_spikes.npy` (50×32×20 uint8) pairs per sample. **These are the files used throughout RTL verification** — never synthetic test vectors.
5. **Model training** (`train_snn.py`, later

`train_snn_with_bias_export.py`): a `TwoLayerNet` — 640→128→2, sigmoid hidden, softmax output — trained via Adam on the *raw rate* (not the spike train itself; mathematically equivalent since firing probability IS the rate). Hidden size (32/64/128) chosen by a sweep, selecting by validation **F2 score** (weights recall over precision — missing a real drone is worse than a false alarm). Final: **128 hidden neurons** confirmed directly from the real exported `.mem` file headers (not just from the training script's own printout, which had a stale docstring still saying "64" — a real, since-resolved point of confusion, see `PROJECT_REPORT.md` for the full story).

6. **CRITICAL, easy to miss:** `train_snn.py` **standardizes** the input feature — $(x - \text{mean}) / \text{std}$, computed once on the training split — before it ever reaches the network. This standardization is NOT applied anywhere in `audio_to_spike.py/validate_encoding.py`'s golden-dataset path (that path stays in raw feature units). **If you ever touch the hidden-layer math, this must stay folded into $w1/b1$ (see §5 below) or accuracy will collapse** — this exact mistake happened once already and dropped real accuracy from ~94% to 60.4% before being caught and fixed. Do not reintroduce it.
7. **Weight export:** `w1, w2, b1, b2` exported as **signed 16-bit two's complement, hex, one value per line, row-major** (`pre_idx*NUM_POST+post_idx` addressing), each matrix with its own independently-computed scale factor (`scale = (215-1)*0.9/max(|w|)`), printed in a `//` comment header line of the `.mem` file. **The biases were originally missing from export entirely** (a real bug in the user's own script, since fixed) — if you're given weight files without matching bias files, that's the same bug recurring, not something to work around silently.

3. The complete hardware pipeline — file-by-file, in dataflow order

All RTL is plain Verilog-2001 (no SystemVerilog in synthesized code — one SV-only construct was caught once in a *testbench*, fixed, never in

synthesizable code). All memories loaded via `$readmemh` at compile time (these are ROMs, not runtime-writable — the whole design is compile-time specialized to whichever samples/weights are baked in via `.mem` file paths).

File	Role	Status
<code>lfsr16.v</code>	16-bit Fibonacci LFSR, bit-exact to Python <code>LFSR16</code>	Verified 2000/2000 vectors bit-exact
<code>threshold_mem.v</code>	ROM: one sample's 640 per-neuron thresholds	Verified
<code>rate_encoder.v</code>	Feature → 50-timestep spike stream. Now supports 4 threshold sets (<code>THRESH_MEM_FILE_0..3</code> params) selected at runtime via <code>sample_sel[1:0]</code> — later addition for the board demo interface, see §6	Verified (single-sample form); 4-way mux form syntax-checked, not yet resimulated (low risk — pure mux addition, doesn't touch verified logic)
<code>weight_mem.v</code>	Generic <code>NUM_PRE × NUM_POST</code> signed-16-bit ROM, reused for <code>w1</code> , <code>w2</code> , and (with <code>NUM_PRE=1</code>) <code>b1/b2</code>	Verified against real files

<code>spike_gated_mac.v</code>	Accumulates <code>w1</code> rows into 128 hidden pre-activations, only for firing neurons — the actual event-driven/spike-gated hardware mechanism	Verified against real 50-timestep data, both samples
<code>combine_scale.v</code>	Rescales <code>raw_acc</code> (<code>w1</code> 's fixed-point units) + <code>b1</code> (its own, DIFFERENT scale) into a common Q9.7 format via two fixed-point multiplies	Verified; constants tied to CURRENT weight files' scale factors — must be recomputed if weights are retrained
<code>sigmoid_lut.v</code>	Fixed-point sigmoid: Q9.7 in, Q0.16 out, 2048-entry half-table (exploits sigmoid symmetry)	Verified against 256 real hidden-neuron values, max diff=0
<code>hidden_neuron.v</code>	Wires <code>spike_gated_mac</code> + <code>b1</code> memory + <code>combine_scale</code> + <code>sigmoid_lut</code> into the complete hidden-layer computation	Verified end-to-end, both samples, first try
<code>output_layer.v</code>	Dense 128-wide dot product ($a1 \cdot w2 + b2$) + argmax. Not spike-gated — <code>a1</code> is already time-integrated, not a spike train	Verified, full pipeline, both samples

<code>snn_top.v</code>	Master FSM: single <code>start/</code> <code>done</code> interface, sequences <code>encode</code> → <code>hidden</code> → <code>output</code> → <code>finalize</code> phases	Verified bit- exact against manual golden values
<code>de2_top.v</code>	Board-level wrapper, NOT used in simulation. Maps <code>KEY[1:0]</code> → 4-way sample selector (auto-triggers classification on key change), <code>LEDG[0]=drone /</code> <code>LEDG[1]=ambient</code> , with a <code>result_valid</code> gate so LEDs stay off before the first real classification	Syntax- checked, not board-tested yet

Two documented "simplest correct first" shortcuts, not yet optimized:

- `spike_gated_mac.v` is fully sequential (one weight read per cycle) — worst case $\sim 640 \times 128$ cycles/timestep. Fine for correctness; a real throughput bottleneck if ever parallelized workloads are needed.
- `snn_top.v`'s wait between changing `hidden_neuron`'s read address and the result being valid is a hardcoded 4-cycle count (`HIDDEN_READ_LATENCY`), not a real valid/ready handshake. Correct as long as `hidden_neuron.v`'s internal pipeline depth doesn't change; brittle if it does.

4. `.mem` files — what's authoritative, what's superseded

Currently authoritative for hardware (use these):

- `W1_folded_input_hidden.mem`, `b1_folded_hidden.mem` — input standardization pre-folded in algebraically ($W1' = W1 / \text{std}$, $b1' = b1 - (\text{mean} / \text{std}) \cdot W1$). Fully derived from and equivalent to the original trained model, just re-expressed so no hardware architecture had to change to support standardization.
- `W2_hidden_output.mem`, `b2_output.mem` — unaffected by the standardization fix (that only touches the input layer), unchanged since first generated.
- `sigmoid_lut.mem` — the 2048-entry fixed-point sigmoid table.
- `threshold_ambient.mem`, `threshold_drone.mem`, `threshold_ambient2.mem`, `threshold_drone2.mem` — 4 real, individually verified audio samples, currently wired to `de2_top.v`'s 4 `KEY[1:0]` codes.

Superseded, do not use for hardware (kept only for historical reference in the report):

- `W1_input_hidden.mem`, `b1_hidden.mem` (the pre-standardization-fold originals).
-

5. Verification methodology and results (the trustworthy part)

Every module was checked bit-exact against an independently-written Python reference, using real audio-derived data, not synthetic test vectors, before being trusted and before the next module was built on top of it. This was not a one-shot check — real bugs were found this way throughout:

- LFSR reset race condition (async reset triggered by a same-cycle register update) — 166/32000 spike bits wrong, found comparing against real spike data, fixed by removing the flawed mid-pass reseed logic.
- `spike_gated_mac.v` double-issuing `h=0` per active row — exactly 2× the correct value on that one index, every other index correct — fixed by not issuing on the row-entry transition.

- `snn_top.v`'s FSM checking `busy` before the submodule had a chance to raise it — over-accumulation (totals too large), fixed by requiring `busy` to be observed high before waiting for it to go low.
- Two separate 32-bit `integer` overflow bugs in testbenches (`tb_output_layer.v`, then `tb_batch_regression.v`) — large golden values silently wrapped in a 32-bit Verilog `integer`; both fixed by widening comparison variables to 64-bit `reg`. If you write a new testbench for this project, default to 64-bit for any golden-value comparison variable — this mistake has recurred once already.
- The big one: missing input standardization (§2.6 above) — found via a real 48-sample batch (not just the original 2 hand-picked samples), accuracy stuck at 60.4%, root-caused via a hidden-neuron saturation check (44% of neurons fully saturated — the signature of wrong input scale), fixed via the algebraic fold, confirmed at actual RTL simulation level (not just Python) — reran the full 48-sample batch through real RTL after the fix: 48/48 bit-exact, 47/48 (97.9%) accuracy vs. ground truth.

Current best verified numbers:

RTL vs. Python golden reference: 48/48 bit-exact (full batch)
 Classification accuracy vs. ground truth: 47/48 (97.9%)

The one "wrong" case in earlier 2-sample testing (a real `bebop` drone clip classified as ambient) was confirmed to be a genuine limitation of the trained model itself (the true float model gets it wrong too, not just the fixed-point hardware) — consistent with the model's own reported ~85.57% drone recall. Don't "fix" this by changing hardware math; it would mean the hardware no longer faithfully represents the trained model.

6. FPGA synthesis status (Quartus II 13.0.1, Cyclone IV EP4CE115)

- 0 errors, full compile flow (Synthesis → Fit → Assembler →

TimeQuest) completed successfully.

- Timing: PASSES 50MHz with positive slack in every corner analyzed (worst case Slow/85°C: +2.046ns setup, +0.386ns hold) — confirmed real once `neuro.sdc` (a clock constraint file, without which TimeQuest defaults to a meaningless unconstrained 1GHz check) was added.
- Resource usage: ~85% of logic cells (97,877/114,480). Fits, with a specific, evidence-based (not yet independently confirmed) hypothesis for the dominant cause: `spike_storage` in `snn_top.v` (the 50×640-bit buffer connecting `rate_encoder`'s streamed output to `hidden_neuron`'s per-timestep input) has a single-bit indexed write pattern (`spike_storage[t][n] <= bit`) that likely defeats Quartus's block-RAM inference, synthesizing as ~32,000 raw flip-flops instead of a few memory blocks. If resource pressure becomes a real problem, this is the first thing to investigate — check Quartus's Compilation Report → Fitter → Resource Usage Summary for `spike_storage`'s specific contribution before assuming this diagnosis is correct.
- One cosmetic, non-blocking synthesis warning was chased through two fix attempts in `combine_scale.v` (Verilog's sized-signed-decimal literal syntax can't directly express -32768 — no `<N>'sd...` form works for the minimum signed value of its own width; the working fix is a plain unsized literal that gets truncated on assignment). If you see `Warning (10259): constant value overflow`, this is almost certainly that same class of issue recurring somewhere — same fix pattern applies.

Board-level demo interface (`de2_top.v`) was added late, after synthesis was already passing, per the user's explicit request:

`KEY[1:0]` selects among 4 pre-loaded real audio samples (no live audio pipeline — this matches the ORIGINAL project brief's explicit allowance to use "already-generated software features/test vectors as the initial FPGA input" rather than building real-time STFT-on-FPGA, which was never in scope). Auto-triggers classification on key-code change; no separate start button. Not debounced (acceptable for a demo — worst case is a harmless re-trigger settling on the same correct answer); no manual reset button (power-on-reset generator used instead).

Not yet done: board-level pin assignment (a verified `.qsf` file was

provided in this same handover package — `de2_top_pins.qsf`), programming the actual board, physical testing.

7. Design decisions made under explicit constraints — know these before changing anything

- **Hybrid hardware approach** (§1): chosen over either "invent an LIF threshold" (no principled derivation possible, since the trained model is sigmoid not ReLU, and the standard ANN-to-SNN conversion theory only covers ReLU) or "fully reconstruct rate and do dense math" (would throw away the genuinely event-driven architecture). The spike-gated accumulate → sigmoid LUT → argmax approach keeps real event-driven hardware while guaranteeing the arithmetic matches the trained model exactly (once standardization was correctly included).
 - **One sample per bitstream / no live audio**: explicit, deliberate scope decision, not a limitation discovered late. Matches the original project brief.
 - **"Simplest correct first"**: both remaining shortcuts (§3) were deliberate choices to get a fully verified, working pipeline before optimizing, per the original brief's stated hackathon priority order ("prioritize getting a reliable working FPGA demonstration first"). Don't "clean these up" reflexively without checking whether they're actually causing a real problem first.
-

8. Remaining work (as of this handover)

1. Assign the pin file (`de2_top_pins.qsf`) in Quartus, or import it.
2. Compile the full flow with `de2_top` as the top-level entity (not `snn_top` — that stays the simulation-facing module).
3. Program the actual DE2-115 board.

4. Test all 4 `KEY[1:0]` codes against known-correct expected results (already known from RTL simulation — should match).
 5. Optional, lower priority: apply the still-pending `combine_scale.v` truncation-warning fix if a fully clean compile log is wanted; verify the `spike_storage` resource hypothesis (§6) if resource headroom becomes an actual constraint; consider debounce/manual-reset additions to `de2_top.v` if the auto-trigger behavior proves flaky on real hardware (untested assumption, not a known problem yet).
 6. If ever asked to run a full RTL sweep of a much larger batch (100+ samples): reuse the `tb_batch_regression.v` pattern (single reused instance pair, runtime `$sformat`-constructed `$readmemh` paths) — don't re-derive this from scratch, and remember the 64-bit golden-value lesson from §5.
-

9. Where everything lives (file manifest)

```
fpga_snn/
├── docs/PROJECT_REPORT.md      ← full session-by-session
├── docs/HANDOVER.md           ← this file
├── quartus/neuro.sdc           ← 50MHz clock constraint
├── quartus/de2_top_pins.qsf    ← verified pin assignment
├── rtl/                        ← all 11 .v files, descriptions
├── tb/                         ← testbenches (simulation)
├── mem/                        ← all .mem files, descriptions
├── sim/                        ← golden reference data
└── python_ref/                 ← Python scripts that generate
```

If continuing this project: read `PROJECT_REPORT.md` for the complete forensic detail behind any specific decision summarized here. This handover file is the fast-start summary; the report is the full record.